

Inteligência Artificial

Redes Neurais artificiais



JeffersonBMartins
Jeffersonbeethm
Por aqui desde maio de 2014
Segue 301 Tem 214 seguidores

+ ADICIONAR AMIGOS

Estatísticas

810 Dias seguidos	57363 Total de XP
Ouro Divisão	3 Pódios

Prof. Dr. Jefferson Beethoven Martins
IFTM Campus UPT – 10/02/2026



Jefferson Beethoven Martins ✓
Engenheiro de Computação - Docente Dr. - Instituto Federal de Educação, Ciência e Tecnologia do Triângulo Mineiro
Uberaba, Minas Gerais, Brasil · [Informações de contato](#)

IFTM - Instituto Federal de Educação, Ciência e Tecnologia do Triângulo Mineiro
Universidade Federal de Uberlândia

Vamos definir, novamente, RNA's

- De forma simplificada, podemos dizer que RNAs são algoritmos do tipo entrada e saída de dados, inspirados no cérebro biológico que são capazes de realizar pelo o menos três tipos de tarefas: **aproximação de funções, reconhecimento de padrões e previsão.**

A natureza nos inspira

- O cérebro é um órgão do corpo biológico incrível. Consegue realizar tarefas **extremamente complexas** com facilidade e perfeição.
- Tarefas que para nós (seres humanos) parecem ser bem simples, demandam **esforço computacional** extremamente complexo e muitas vezes imperfeito quando simulados pelo computador.
- **Reconhecer rostos**, perceber movimentos, gerenciar movimentos do corpo para tocar violão, teclado ou outro instrumento, **tomar decisões**... E claro, fazer muitas tarefas ao mesmo tempo que gerencia diversas funções vitais do corpo humano.
- O cérebro biológico possui uma capacidade extremamente poderosa de **aprendizagem**: por experiência, observação, por cognição, etc.

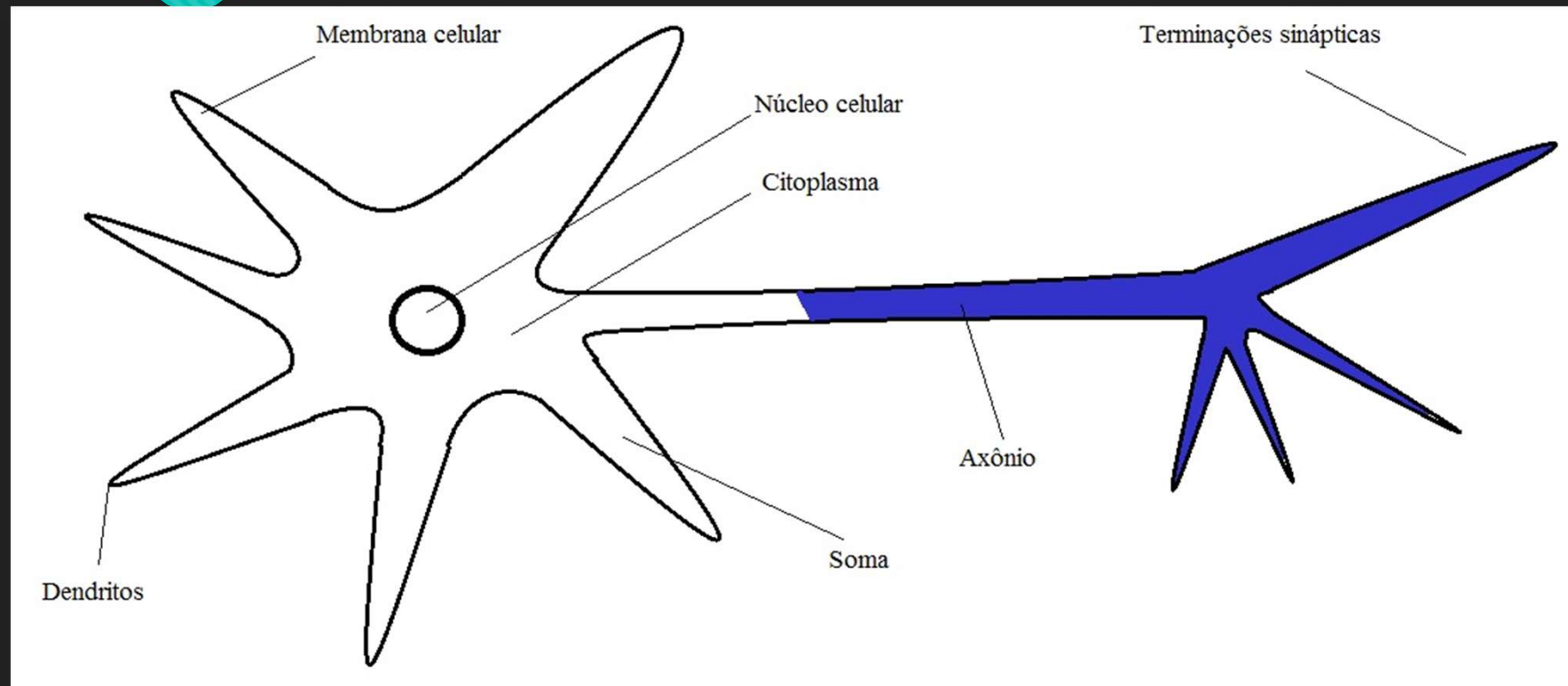
Mistérios do cérebro

- A massa do cérebro é de aproximadamente 1,5 kg.
- Consome cerca de 20% da energia do corpo.
- Cada neurônio tem comprimento aproximado de 100 μm .
- Estima-se que o cérebro humano tenha 100 bilhões de neurônios.
- Estima-se que cada neurônio seja interligado a outros 6000 neurônios.

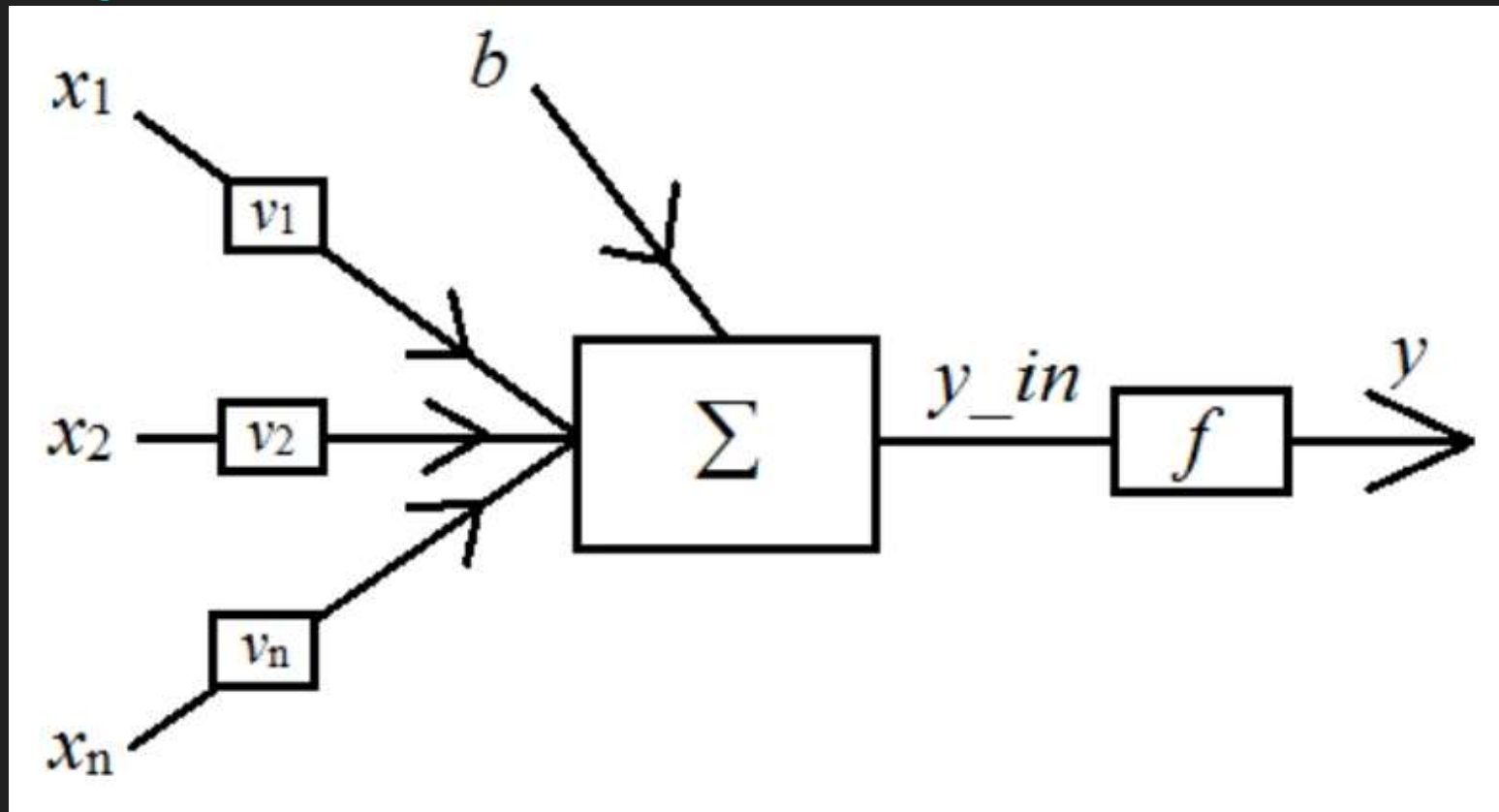
A arquitetura da máquina perfeita

- A célula elementar do sistema nervoso central é o neurônio e seu papel consiste em conduzir impulsos (estímulos elétricos advindos de reações físico-químicas).
- É basicamente dividido em três partes principais: dendritos, corpo celular (membrana celular, citoplasma, núcleo celular e soma) e axônio.
- Os dendritos são responsáveis por captar os estímulos vindos de outros neurônios ou do próprio meio externo.
- **O corpo celular processa as informações recebidas e produz um potencial de ativação que poderá disparar um impulso elétrico para o axônio.**
- **O axônio conduz o impulso elétrico para outros neurônios conectores.**

Olhando fica mais fácil



O neurônio artificial



Onde tudo começou

- Proposto por McCulloch & Pitts (1943), a figura anterior ilustra o modelo de neurônio artificial mais simples.
- De maneira análoga ao cérebro biológico, os sinais de entrada $x_1, x_2, \dots, x_n$ são análogos aos impulsos elétricos do meio externo.
- As ponderações exercidas pelas terminações sinápticas do modelo biológico, são representadas no neurônio artificial pelos pesos sinápticos $v_1, v_2, \dots, v_n$.
- De modo análogo ao neurônio biológico, a relevância da informação processada é dada pela multiplicação de x_i por v_i .
- **Assim, a saída artificial do neurônio é a soma ponderada de suas entradas.**

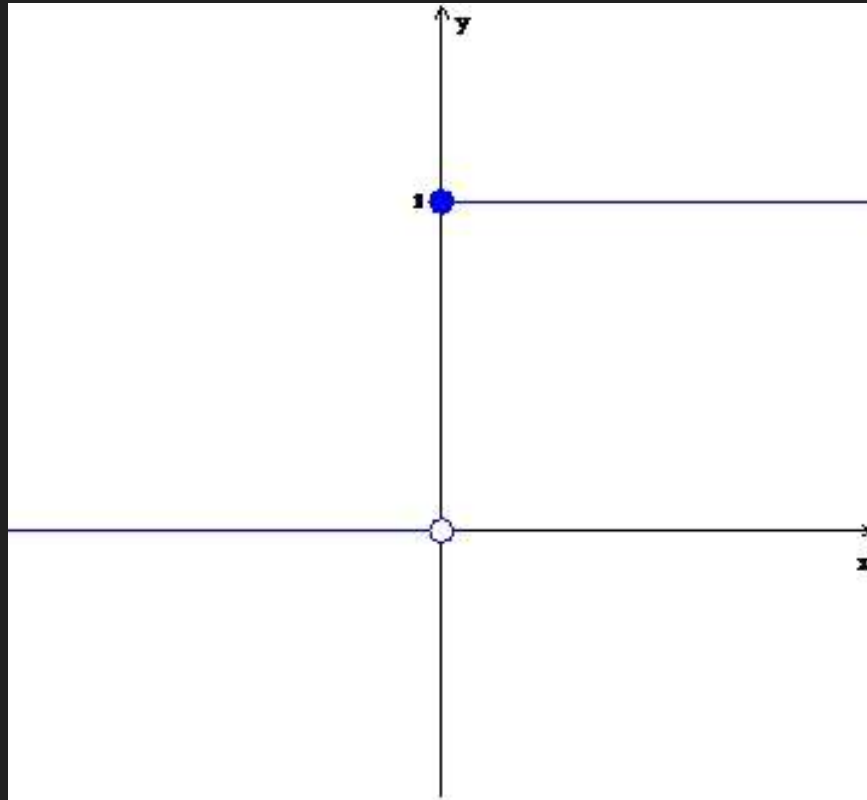
Jefferson, explique melhor...

1. **Sinais de entrada** $\{x_1, x_2, \dots, x_n\}$: Usualmente normalizados, são valores numéricos advindos do problema que está sendo resolvido.
2. **Pesos sinápticos** $\{v_1, v_2, \dots, v_n\}$: São valores responsáveis por ponderar as entradas e determinar sua relevância em relação aquele neurônio.
3. **Combinação Linear** Σ : É responsável por agregar todos os sinais ponderados pelos pesos de forma a produzir o potencial de ativação.
4. **Limiar de ativação** b : É responsável por determinar o quão apropriado é o resultado da combinação linear em gerar um disparo em direção a saída do neurônio.
5. **Potencial de ativação** y_{in} : É a soma entre a combinação linear e o limiar de ativação. Se o resultado é positivo, dizemos que há um potencial excitatório. Caso contrário, dizemos que há um potencial inibitório.
6. **Função de ativação** f : Limita a saída do neurônio dentro de um intervalo específico.
7. **Sinal de saída** y : Valor final de saída a ser levado a saída da RNA ou a outro neurônio.

Ainda não está claro...

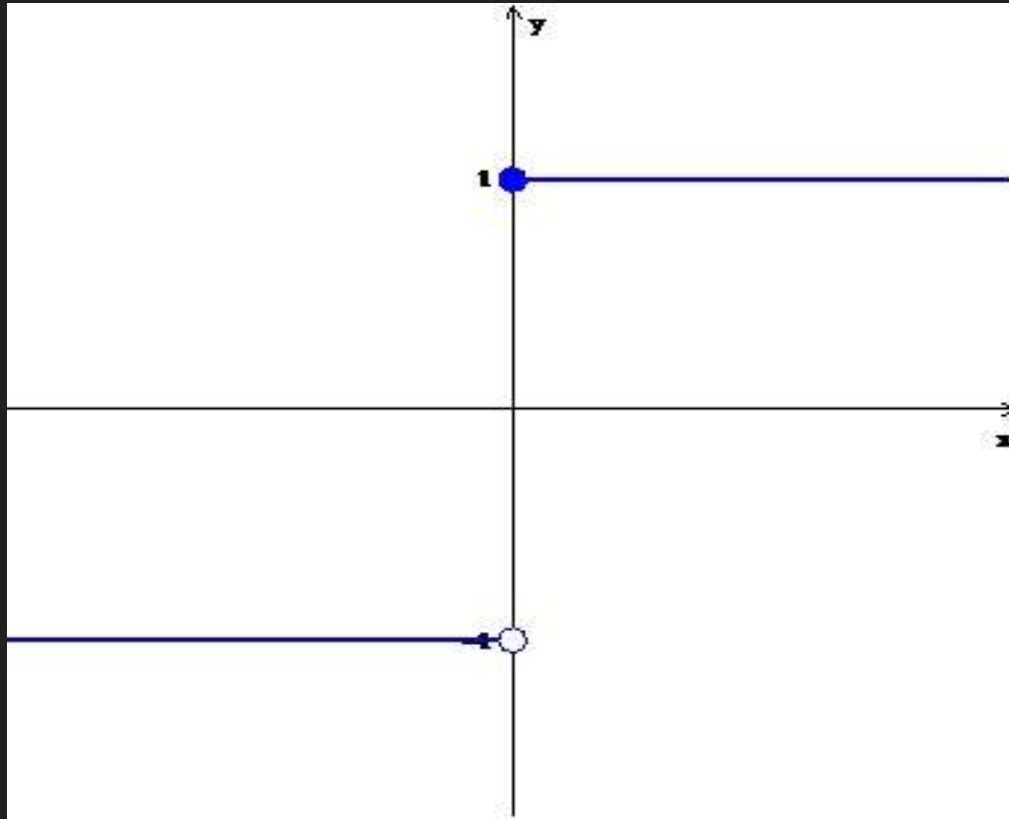
1. O conjunto de valores que representam as variáveis de entrada são apresentados.
2. Cada entrada é multiplicada pelo respectivo peso sináptico.
3. Realiza-se a somatória de todos os produtos a fim de produzir o potencial de ativação.
4. Adiciona-se o limiar ao potencial de ativação.
5. Aplica-se a função de ativação .
6. Propaga-se o resultado pela saída do neurônio.

Funções de ativação - degrau



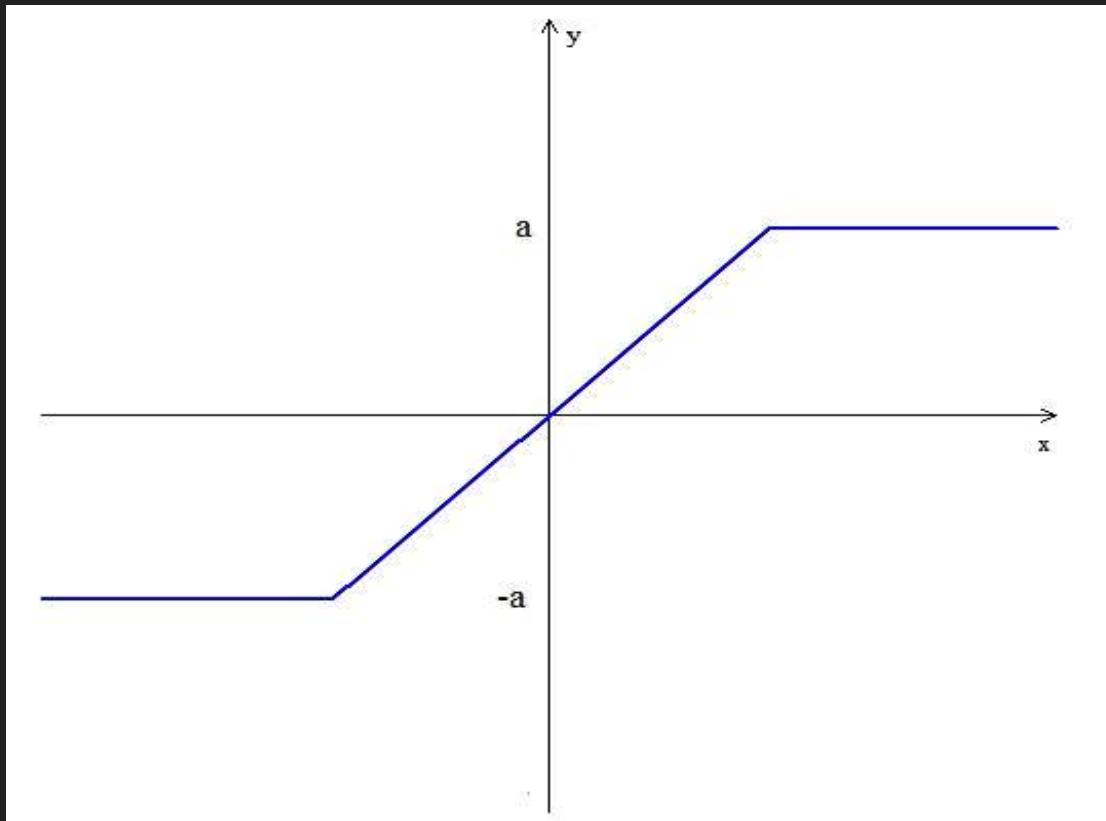
$$f(x) = \begin{cases} 1 & \text{se } x \geq 0 \\ 0 & \text{se } x < 0 \end{cases}$$

Funções de ativação – degrau bipolar



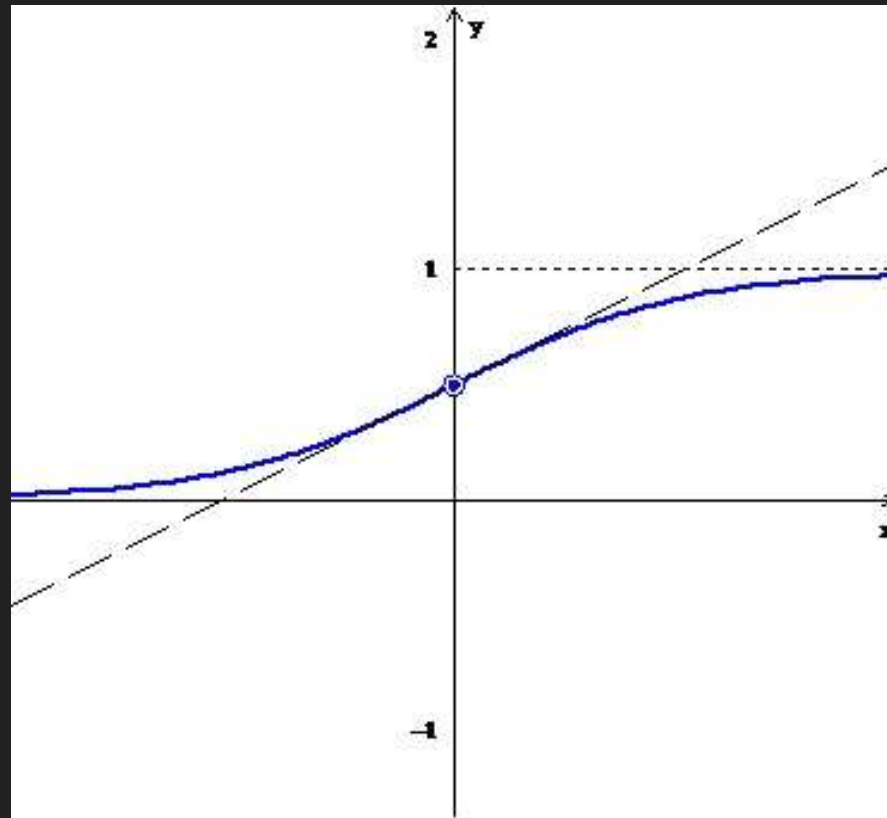
$$f(x) = \begin{cases} 1 & \text{se } x \geq 0 \\ -1 & \text{se } x < 0 \end{cases}$$

Funções de ativação – degrau rampa



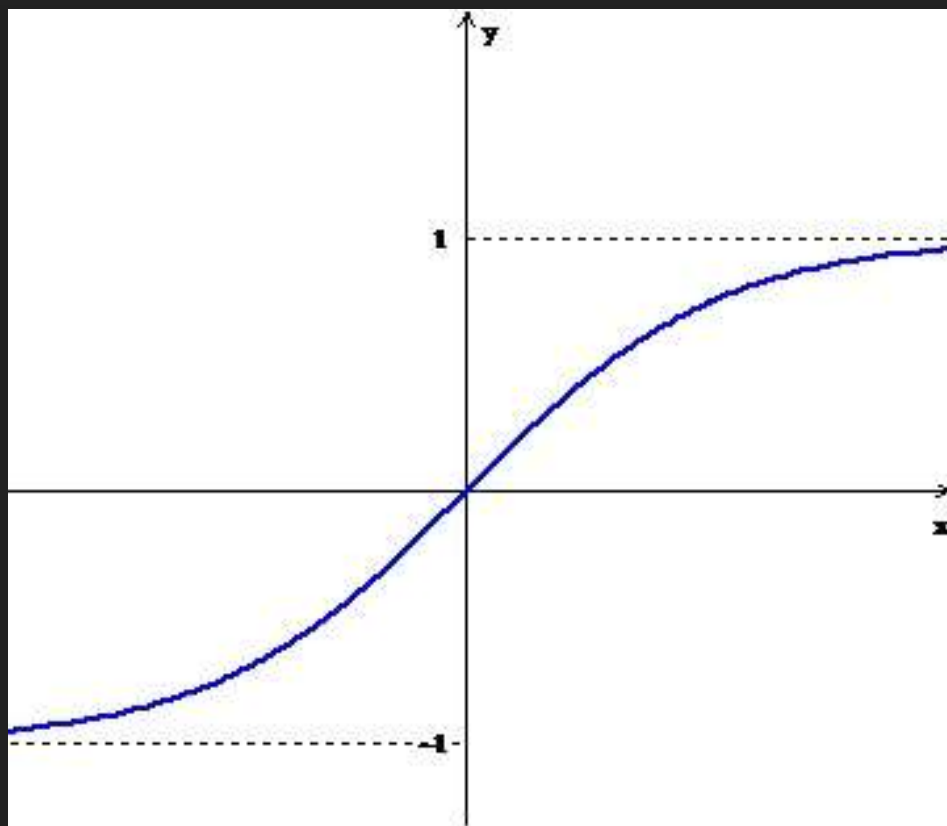
$$f(x) = \begin{cases} -a & \text{se } x < -a \\ x & \text{se } -a \leq x \leq a \\ a & \text{se } x > a \end{cases}$$

Funções de ativação – logística



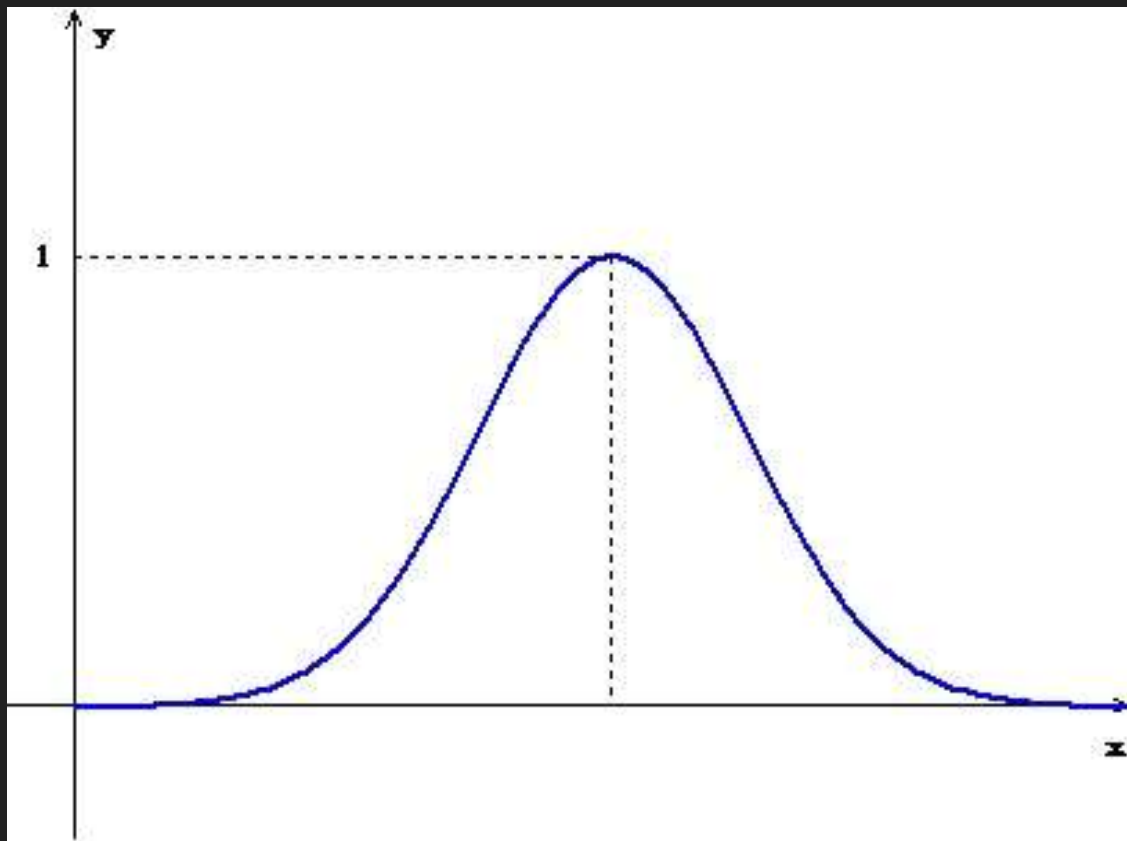
$$f(x) = \frac{1}{1 + e^{-\beta \cdot x}}$$

Funções de ativação – tangente hiperbólica



$$\frac{e^{2x} - 1}{e^{2x} + 1}$$

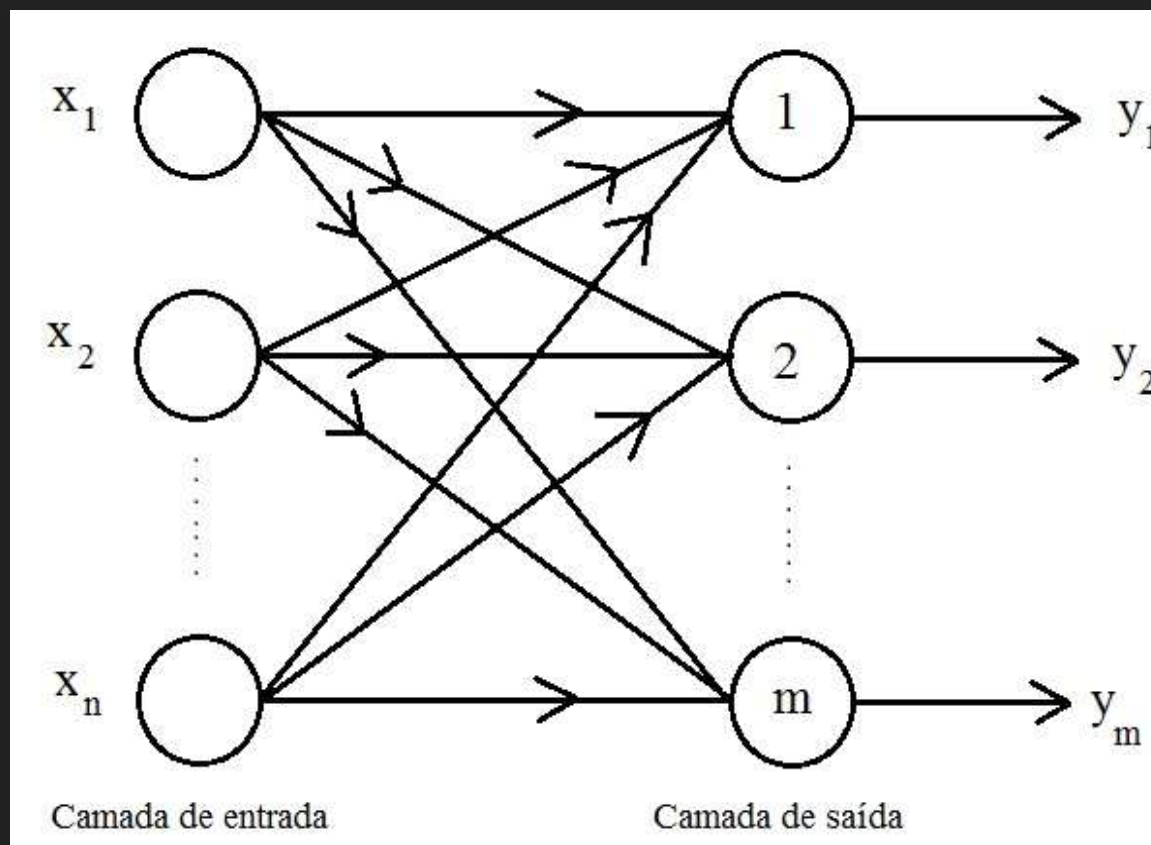
Funções de ativação – gaussiana



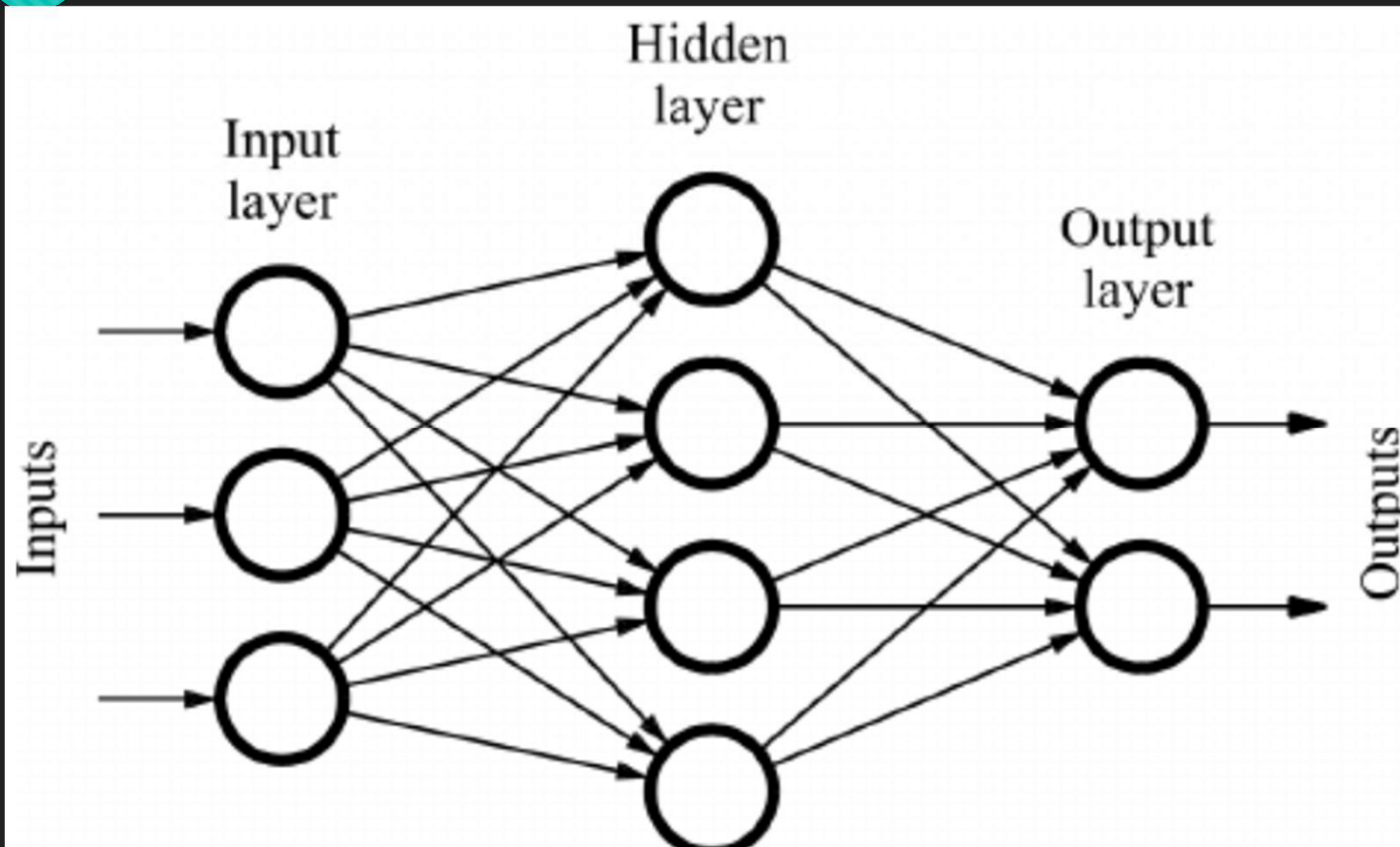
$$f(x) = a \cdot e^{-\frac{(x-b)^2}{2c^2}}$$

O parâmetro a é a altura do pico da curva, b é a posição do centro do pico, e c controla a largura do "sino".

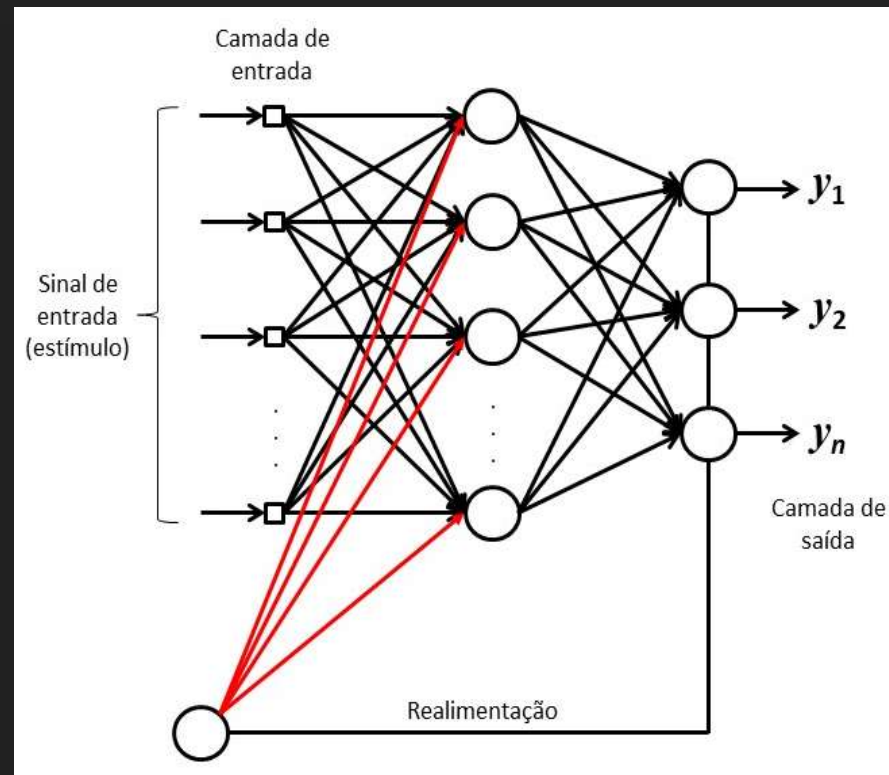
Arquitetura feedforward de camada simples



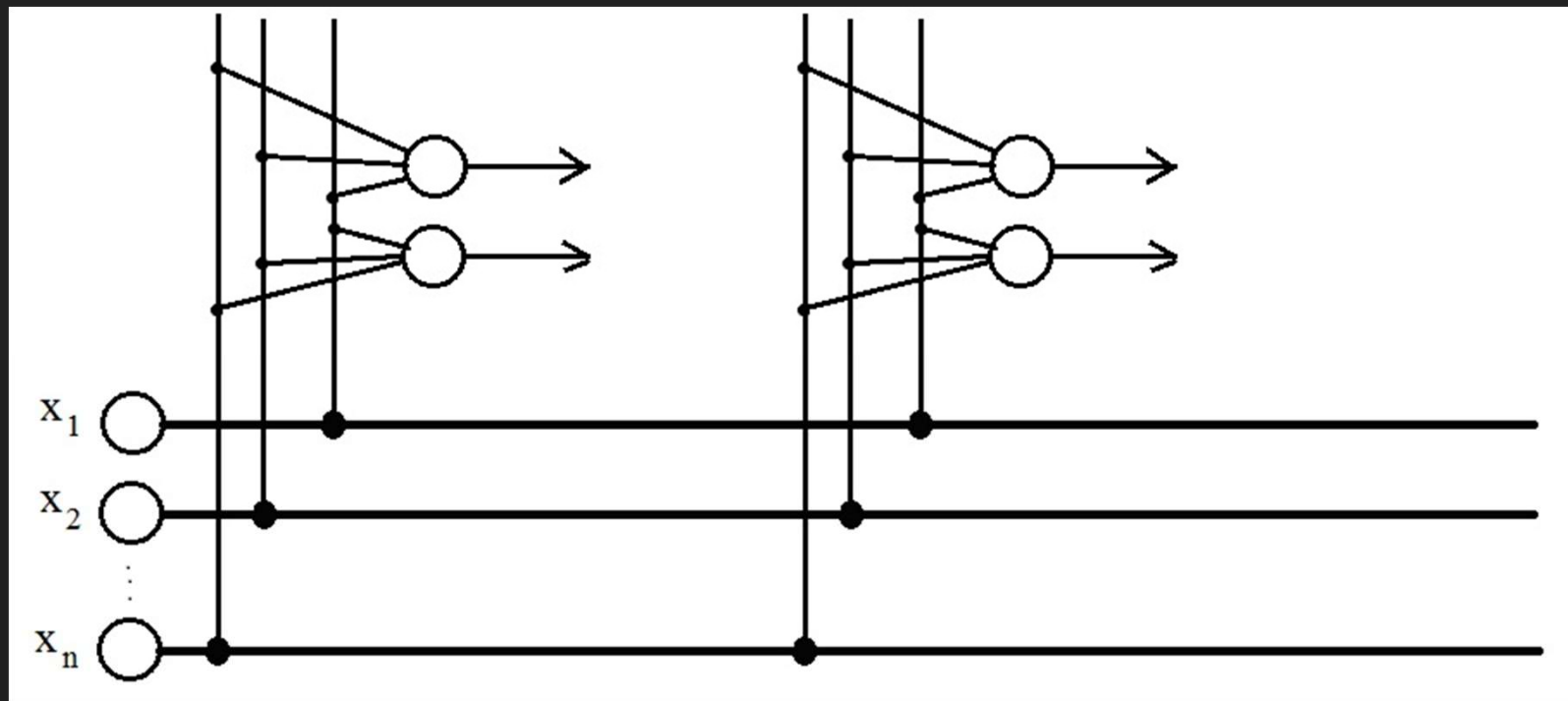
Arquitetura feedforward de camadas múltiplas



Arquitetura recorrente ou realimentada



Arquitetura reticulada



O treinamento de RNAs

- RNAs são algoritmos que levam em consideração dados amostrais nela inseridos e, de forma iterativa, ajustam seus pesos sinápticos para que seja possível realizar a tarefa desejada.
- Esse processo é denominado como treinamento. Assim como o cérebro biológico precisa de treinamento para realizar determinadas tarefas, também é preciso treinar as RNAs.
- Há basicamente três etapas para uma RNA
 1. Treinamento (60 a 90% dos dados)
 2. Teste (10 a 40% dos dados)
 3. Operação

O treinamento supervisionado

- Trata-se de uma estratégia de treinamento aplicável quando se conhece amostras que possuem os dados de entrada e a respectiva saída desejada.
- Ao longo do treinamento as entradas são inseridas na rede e as saídas produzidas são comparadas com as saídas desejadas (targets).
- A diferença entre as saídas produzidas e desejadas é utilizada para o ajuste dos pesos sinápticos.
- O treinamento pode durar por um número de ciclos (épocas ou iterações) pré-determinado ou até o alcance de um erro estabelecido.
- Denomina-se por ciclo a apresentação de todas as amostras de treinamento. Em outras palavras, a cada ciclo, cada amostra de treinamento é apresentada uma vez como entrada da rede.

Treinamento por lotes *off-line*

- Se enquadra dentro da categoria de treinamento supervisionado.
- No treinamento por lotes *off-line*, o ajuste dos pesos sinápticos ocorre somente após a apresentação de todas as amostras do conjunto de treinamento.
- Cada passo de ajuste leva em consideração todos os desvios observados pelas saídas produzidas e saídas desejadas.
- Em outras palavras há o ajuste dos pesos por ciclo.

Treinamento por lotes *on-line*

- Também se enquadra dentro da categoria de treinamento supervisionado.
- No treinamento por lotes *on-line*, o ajuste dos pesos sinápticos ocorre após a apresentação de cada amostra do conjunto de treinamento.
- **É aplicável a problemas onde o comportamento do sistema varia com bastante rapidez.**

O treinamento não-supervisionado

- É aplicável a problemas onde existem os dados de entrada, mas não existem as saídas desejadas.
- O propósito do treinamento é fazer com que a rede se organize a fim de estabelecer conjuntos (*clusters*) de elementos da amostra que contenham características semelhantes. Alguns autores definem como processo de *clusterização*.
- A quantidade de clusters pode ser um parâmetro estabelecido para o treinamento ou ainda, pode ser determinado pela própria rede.

Historicamente, onde estamos?

- A primeira publicação relacionada a neurocomputação é de 1943. MCCulloch & Pitts realizaram o primeiro modelamento matemático inspirado no neurônio biológico, o que corresponde ao primeiro neurônio artificial.
- Em 1949, Hebb desenvolveu o primeiro método de treinamento para RNAs, denominado como Regra de Aprendizagem de Hebb.

Regra de aprendizado de Hebb?



A regra de aprendizado de Hebb propõe que o peso de uma conexão sináptica deve ser ajustado se houver sincronismo entre os níveis de atividade das entradas e saídas [Hebb, 1949].

Os princípios de Hebb

- Neurônios que disparam juntos permanecem juntos!
- Conexão forte: peso alto;
- Conexão fraca: peso baixo;
- Este algoritmo é “não supervisionado”;
- O importante é “**associar**”.

O papel de Hebb

- Ele é a base das RNA's;
- Os pesos podem crescer ou decrescer indefinidamente;
- Não resolve problemas complexos;
- Há um reforço de correlações;

Traduzindo

- Os neurônios são unidades computacionais;
- As sinapses são os pesos;
- O aprendizado surge quando alteramos os pesos;
- Quando aprendemos algo e repetimos, há um reforço no nosso cérebro.

Vamos ao código - 01

```
1  #include <stdio.h>
2  //Adaptado do Gemini.
3  int main()
4  {
5      int inputs[4][2] = {
6          {1, 1},
7          {1, -1},
8          {-1, 1},
9          {-1, -1}
10     };
11     int targets[4] = {1, -1, -1, -1};
12
13     float w1 = 0, w2 = 0, bias = 0;
14
15     printf("--- Iniciando Treinamento (Regra de Hebb) ---\n");
16
17     for (int i = 0; i < 4; i++)
18     {
19         int x1 = inputs[i][0];
20         int x2 = inputs[i][1];
21         int y = targets[i];
22
23         // Regra de Hebb: W = x * y
24         w1 = w1 + (x1 * y);
25         w2 = w2 + (x2 * y);
26         bias = bias + y;
27
28         printf("Amostra %d: Pesos atualizados -> W1: %.1f, W2: %.1f, Bias: %.1f\n", i+1, w1, w2, bias);
29     }
30 }
```

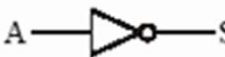
Vamos ao código - 02

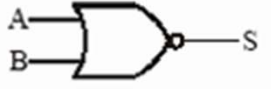
```
30
31 printf("\n--- Teste Final ---\n");
32 for (int i = 0; i < 4; i++) {
33     float soma = (inputs[i][0] * w1) + (inputs[i][1] * w2) + bias;
34     int predicacao = 0;
35     if(soma >= 0){
36         predicacao = 1;
37     }else{
38         predicacao = -1;
39     }
40
41     printf("Entrada: [%2d, %2d] | Alvo: %2d | Predicao: %2d\n", inputs[i][0], inputs[i][1], targets[i], predicacao);
42 }
43
44 return 0;
45 }
```


Essa rede é limitada, mas separa bem!

- Teste para todas as portas lógicas (trabalho 01);
- Não se esqueçam, todos os trabalhos devem ser apresentados em sala.
- Mande o trabalho por e-mail: jefferson@iftm.edu.br (recomendo fazer um zip com "rar" e senha).

Essa rede é limitada, mas separa bem!

E AND		<table><tr><th>A</th><th>B</th><th>S</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	S	0	0	0	0	1	0	1	0	0	1	1	1
A	B	S															
0	0	0															
0	1	0															
1	0	0															
1	1	1															
OU OR		<table><tr><th>A</th><th>B</th><th>S</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	S	0	0	0	0	1	1	1	0	1	1	1	1
A	B	S															
0	0	0															
0	1	1															
1	0	1															
1	1	1															
NÃO NOT		<table><tr><th>A</th><th>S</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	S	0	1	1	0									
A	S																
0	1																
1	0																
NE NAND		<table><tr><th>A</th><th>B</th><th>S</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	S	0	0	1	0	1	1	1	0	1	1	1	0
A	B	S															
0	0	1															
0	1	1															
1	0	1															
1	1	0															

NOU NOR		<table><tr><th>A</th><th>B</th><th>S</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	S	0	0	1	0	1	0	1	0	0	1	1	0
A	B	S															
0	0	1															
0	1	0															
1	0	0															
1	1	0															
OU Exclusivo		<table><tr><th>A</th><th>B</th><th>S</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	S	0	0	0	0	1	1	1	0	1	1	1	0
A	B	S															
0	0	0															
0	1	1															
1	0	1															
1	1	0															
Coincidência		<table><tr><th>A</th><th>B</th><th>S</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	S	0	0	1	0	1	0	1	0	0	1	1	1
A	B	S															
0	0	1															
0	1	0															
1	0	0															
1	1	1															

Referências e agradecimentos

- HAYKIN, Simon. Redes neurais: princípios e prática. Bookman Editora, 2007.
- FAUSETT, Laurene. Fundamentals of neural networks: architectures, algorithms, and applications. Prentice-Hall, Inc., 1994.
- Silva, I. N., Spatti, D. H., & Flauzino, R. A. (2010). Redes neurais artificiais para engenharia e ciências aplicadas curso prático. Artliber.
- Agradecimentos especiais ao professor José Ricardo Gonçalves Manzan pela doação do material.